Finding the Ceil of a Target in a Sorted Array

using Pointers and Binary Search

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

1. Problem Statement

Write a C program that reads the size and the elements of a sorted integer array arr[] (ascending order) of variable length, followed by an integer target. The array must be passed to a function named ceil_index() using its address, and pointers must be used to access the array in both the calling function and in ceil_index().

The function ceil_index() must:

- Find the index of the smallest element in arr[] that is greater than or equal to the target.
- Treat this element as the ceil of the target.
- If the ceil element occurs more than once, return the index of its first occurrence.
- Display the ceil element along with its index.
- Display -1 if no element in the array is greater than or equal to the target.

2. Approach and Concepts Used

Since the array is already sorted in ascending order, a linear scan is not required. A modified Binary Search is used, which finds the answer in O(log n) time.

- Variable-length array: the size n is read at run time and memory is allocated dynamically using malloc().
- Pointers: the array is passed to ceil_index() as an int * (its base address). Elements are accessed with pointer notation such as *(arr + mid), and input is read using scanf("%d", arr + i).
- Binary search (lower bound): whenever arr[mid] >= target, mid is stored as a candidate answer and the search continues in the left half to look for an earlier occurrence. Otherwise the search moves to the right half.
- First occurrence: because the search always keeps moving left after finding a valid element, the final stored candidate is the first index holding the ceil value.

3. Algorithm

1. Start.
2. Read the size n and allocate memory for n integers.
3. Read the n sorted elements and the integer target.
4. Call ceil_index(arr, n, target), passing the base address of the array.
5. In ceil_index(): set low = 0, high = n - 1 and result = -1.
6. While low <= high: compute mid = low + (high - low) / 2.
7. If *(arr + mid) >= target, set result = mid and high = mid - 1; otherwise set low = mid + 1.
8. After the loop, if result is -1 display -1; otherwise display the ceil element *(arr + result) and its index result.

9. Free the allocated memory.
10. Stop.

4. Source Code

```
#include <stdio.h>
#include <stdlib.h>

/* Finds the ceil of target in a sorted (ascending) array using pointers.
   Prints the result and returns the index of the first occurrence
   of the ceil element, or -1 if no element is >= target. */
int ceil_index(int *arr, int n, int target)
{
    int low = 0, high = n - 1, mid;
    int result = -1;

    while (low <= high)
    {
        mid = low + (high - low) / 2;

        if (*(arr + mid) >= target)
        {
            result = mid;       /* possible answer, look for an earlier one */
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    if (result == -1)
        printf("-1\n");
    else
        printf("Ceil of %d: %d at index %d\n",
               target, *(arr + result), result);

    return result;
}

int main()
{
    int n, i, target;
    int *arr;

    printf("Enter the size of the array: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid array size.\n");
        return 1;
    }

    arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Enter %d elements in ascending order: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", arr + i);

    printf("Enter the target: ");
    scanf("%d", &target);

    ceil_index(arr, n, target);

    free(arr);
    return 0;
}
```

5. Sample Input and Output

The program was compiled with gcc and tested on the three given examples. Text after each prompt is the value entered by the user.

Test Case 1: target lies between elements, duplicates present

```
Enter the size of the array: 6
Enter 6 elements in ascending order: 5 7 7 8 8 10
Enter the target: 6
Ceil of 6: 7 at index 1
```

Explanation: 7 is the smallest element that is >= 6. It occurs at indices 1 and 2, so the first occurrence (index 1) is reported.

Test Case 2: target is present in the array

```
Enter the size of the array: 6
Enter 6 elements in ascending order: 5 7 7 8 9 10
Enter the target: 8
Ceil of 8: 8 at index 3
```

Explanation: 8 itself is present at index 3, so it is its own ceil.

Test Case 3: no element is greater than or equal to the target

```
Enter the size of the array: 6
Enter 6 elements in ascending order: 5 7 7 8 8 10
Enter the target: 11
-1
```

Explanation: the largest element is 10, which is less than 11, so -1 is displayed.

6. Summary of Test Cases

No.	Array	Target	Output	Result
1	5 7 7 8 8 10	6	Ceil of 6: 7 at index 1	Pass
2	5 7 7 8 9 10	8	Ceil of 8: 8 at index 3	Pass
3	5 7 7 8 8 10	11	-1	Pass
4	4	2	Ceil of 2: 4 at index 0	Pass

7. Complexity Analysis

- Time complexity: O(log n), because the search range is halved in every iteration.
- Space complexity: O(1) extra space, apart from the input array itself.

8. Conclusion

The program correctly finds the ceil of a target value in a sorted array of user-defined size. By combining dynamic memory allocation, pointer-based array access and a modified binary search, it handles duplicate values (returning the first occurrence), exact matches, and the case where no ceil exists. The results for all the tested inputs match the expected outputs.